Iteration I3: Limiting Access

We want to prevent people without an administrative login from accessing our site's admin pages. It turns out that we can do it with very little code using the Rails *callback* facility.

Rails callbacks allow you to intercept calls to action methods, adding your own processing before they're invoked, after they return, or both. In our case, we'll use a *before action* callback to intercept all calls to the actions in our admin controller. The interceptor can check `session[:user_id]`. If it's set and if it corresponds to a user in the database, the application knows an administrator is logged in and the call can proceed. If it's not set, the interceptor can issue a redirect, in this case to our login page.

Where should we put this method? It could sit directly in the admin controller, but—for reasons that'll become apparent shortly—let's put it instead in `ApplicationController`, the parent class of all our controllers. This is in the `application_controller.rb` file in the `app/controllers` directory. Note, too, that we chose to restrict access to this method. This prevents it from ever being exposed to end users as an action:

rails7/depot_r/app/controllers/application_controller.rb

```
  class ApplicationController < ActionController::Base
»   before_action :authorize

    # ...
»
»   protected
»
»     def authorize
»       unless User.find_by(id: session[:user_id])
»         redirect_to login_url, notice: "Please log in"
»       end
»     end
  end
```

The `before_action` line causes the `authorize` method to be invoked before every